

WEF1VO Web Fundamentals

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

WS 2025/26

Vorlesung 4

CSS Box-Modell & Responsive Web

4.1 Das CSS Box-Modell

Das *CSS Box-Modell* [7, Kapitel 8; 8] definiert die Berechnung der gesamten Breite und Höhe von HTML-Elementen.

4.1.1 Überblick

In HTML gibt es eine Reihe von Elementen, die standardmäßig einen neuen Absatz erzeugen und deren Darstellung somit als Block, d. h. als Rechteck geschieht. Vor HTML5 wurden diese als Blockelemente bezeichnet, nun gehören sie zu den Kategorien Flow Content, Sectioning Content und Heading Content. Typische Beispiele sind etwa `<p>`, `<div>` oder auch Überschriften und Gliederungselemente. Mit Hilfe des CSS Box-Modells wird bestimmt, welche Dimension ein solches Blockelement hat. Die Gesamtgröße ist dabei eine Addition von vier Werten:

- Breite (**width**) oder Höhe (**height**) des Elements (je nachdem, ob die horizontale oder vertikale Größe berechnet werden soll). Sie definieren die Größe des Inhaltsbereichs.
- Innenabstand des Elements zu seinem Rand bzw. zu dessen Rahmen (**padding**).
- Rahmenstärke (**border-width**).
- Außenabstand vom Rahmen zum nächsten Element (**margin**).

Abbildung 4.1 zeigt ein Beispiel. Ein Blockelement (z. B. `<div>`) erhält eine Breitenangabe von 90px. Darüber hinaus einen Innenabstand von 10px, einen 5px dicken Rahmen, sowie einen Außenabstand von 20px. Obwohl das Element mit 90px Breite definiert wurde, beträgt sein gesamter horizontaler Platzbedarf nun 160px, denn zu den 90px Breite kommen zwei Mal (links und rechts) 10px für den Innenabstand, zwei Mal 5px für die Rahmenstärke und zwei Mal 20px für den Außenabstand zu angrenzenden Elementen hinzu. Für die Höhe wurden 40px definiert. Zusammen mit denselben Werten für Innenabstand, Rahmen und Außenabstand ergibt sich eine Gesamthöhe von 110px.

In den folgenden Abschnitten wird auf die einzelnen CSS-Eigenschaften eingegangen, die bei der Berechnung des Box-Modells eine Rolle spielen.

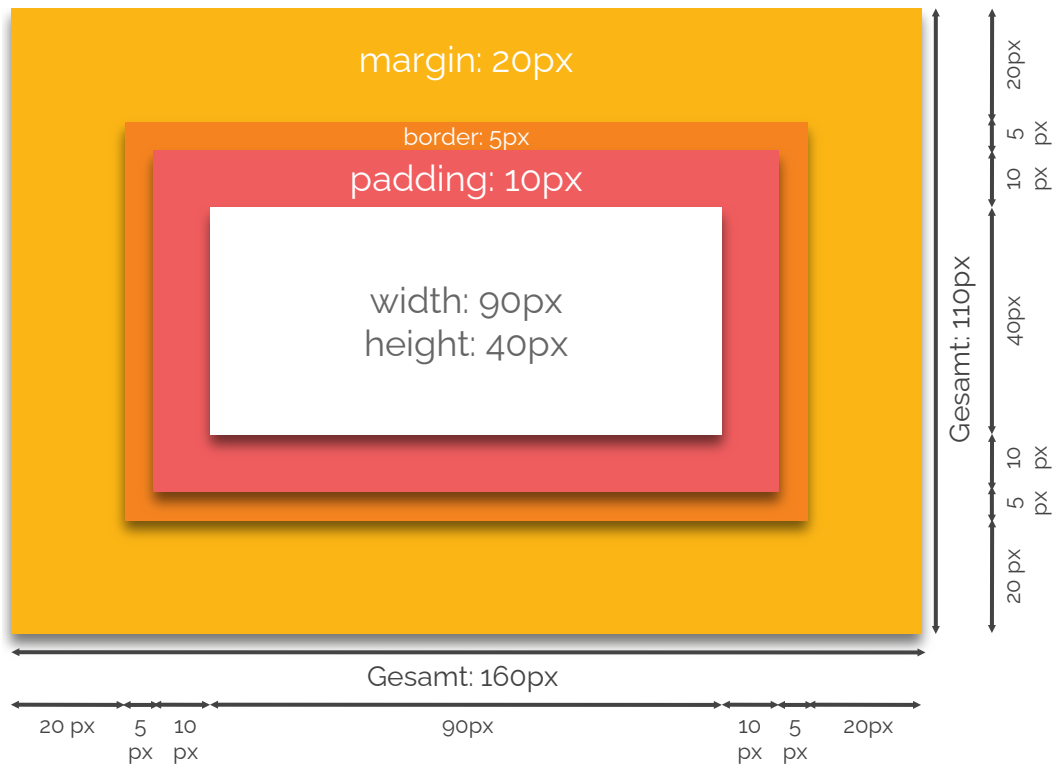


Abbildung 4.1: Die Gesamtbreite eines Objekts mit 90 Pixel Breite, einem Innenabstand von 10 Pixeln, einem Rahmen mit Stärke 5 Pixel und einem Außenabstand von 20 Pixeln ergibt 160 Pixel. Die Gesamthöhe bei 40 Pixel Breite und denselben Angaben für Innen- und Außenabstand sowie Rahmen beträgt 110 Pixel.

4.1.2 Dimensionsangaben für den Inhalt

Um die Dimensionen von Elementen, d. h. ihren Inhaltsbereich anzugeben, sind sowohl fixe als auch Mindest- bzw. Maximalangaben möglich.

Fixe Angaben

Um die *Breite* eines Elements anzugeben, wird die Eigenschaft `width` verwendet. Sie akzeptiert numerische Werte mit den üblichen Angaben. Wird eine Prozentangabe verwendet, bezieht sich diese auf die Größe des einschließenden Elements (im äußersten Fall das Browserfenster).

Um die *Höhe* eines Elements festzusetzen, wird `height` verwendet. Diese Eigenschaft funktioniert analog zur Breitenangabe.

Mit den folgenden CSS-Angaben werden zwei `<div>`-Elemente über ID-Selektoren mit jeweils unterschiedlicher Breite und Höhe definiert, das Ergebnis ist in Abbildung 4.2 ersichtlich.

```
1 #one {
2   width: 150px;
3   height: 50px;
4   background-color: orange;
```

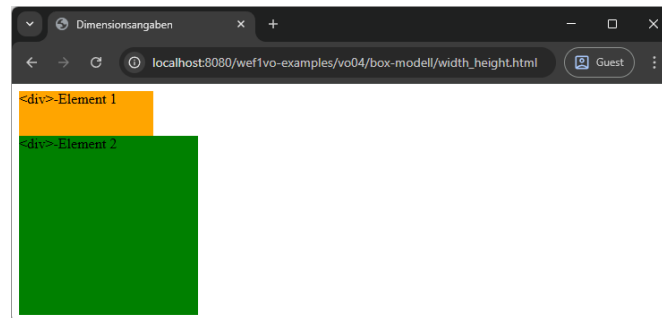


Abbildung 4.2: Die beiden `<div>`-Elemente nehmen genau die Größe ein, mit der sie definiert wurden. Ohne Breitenangabe etwa, würden die Elemente die gesamte Browserfensterbreite beanspruchen.

```
5 }  
6 #two {  
7   width: 200px;  
8   height: 200px;  
9   background-color: green;  
10 }
```

Werden die Angaben zu Breite und Höhe nicht angegeben (dies entspricht auch dem Setzen des Werts `auto`) nimmt das Element so viel horizontalen Platz ein, wie möglich. Ist es von keinem Element umgeben, beansprucht es die gesamte Browserfensterbreite, ansonsten die gesamte Breite des umgebenden Elements.

Wird die Höhe nicht angegeben (oder auf den Wert `auto` gesetzt), so nimmt das Element jedoch die *minimale* Höhe ein. D. h. es wird so weit „zusammengestaucht“, wie möglich.

Egal ob Höhe und Breite angegeben wurden oder nicht, ein Blockelement beansprucht immer eine eigene „Zeile“, d. h. es stehen (ohne zusätzliche Angaben) nie zwei derartige Elemente nebeneinander, auch wenn Platz wäre.

Im folgenden, leicht modifizierten Code, wurden die Dimensionsangaben für das erste `<div>`-Element auf `auto` gesetzt, für das zweite komplett weggelassen. Das Ergebnis ist dasselbe und in Abbildung 4.3 zu sehen.

```
1 #one {  
2   width: auto;  
3   height: auto;  
4   background-color: orange;  
5 }  
6 #two {  
7   background-color: green;  
8 }
```

Mindest- und Maximalangaben

Mit den Eigenschaften `min-width` und `max-width` bzw. `min-height` und `max-height` lässt sich definieren, wie breit bzw. hoch ein Element *mindestens* oder *maximal* sein darf. In Abbildung 4.3 ist ersichtlich, dass ein Element nur so viel Höhe einnimmt,

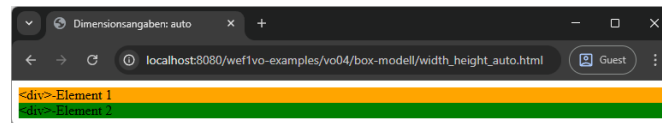


Abbildung 4.3: Ohne Breiten- und Höhenangaben (bzw. mit der Angabe `auto`), nehmen die Elemente so viel horizontalen und so wenig vertikalen Platz ein, wie möglich.

wie der Inhalt dies verlangt. Das kann jedoch zu unerwünschten Effekten führen, vor allem, wenn zum Erstellungszeitpunkt der Seite nicht bekannt ist, wie viel Inhalt ein Bereich aufweisen wird. Soll in diesem Fall verhindert werden, dass ein Element zu niedrig wird, kann die Eigenschaft `min-height` verwendet und das Element damit auf z. B. mindestens 50px gesetzt werden. Somit ist es egal, ob kein Inhalt oder auch nur eine Zeile im Element enthalten sind – dieses wird immer mindestens 50px Höhe aufweisen. Ist jedoch mehr Inhalt vorhanden (der mehr als 50px in der Vertikalen benötigt), so wird das Element normal auf seinen Platzbedarf erweitert.

Um nun auch zu bewirken, dass ein Element nicht höher als z. B. 200px werden darf, kann zusätzlich noch die Eigenschaft `max-height` angegeben werden und somit die Höhe auf 200px begrenzt werden – egal wie viel Inhalt sich im Element befindet. Das Element skaliert nun variabel zwischen 50px und 200px.

Die Eigenschaften `min-width` und `max-width` ermöglichen dasselbe Verhalten auf horizontaler Ebene.

Überlauf bei übergroßem Inhalt

Wird einem Element eine fixe oder auch maximale Höhe und/oder Breite definiert, kann es vorkommen, dass der Inhalt (Text oder auch eine Grafik) zu groß für den definierten Bereich ist. In diesem Fall kann mit Hilfe der Eigenschaft `overflow` bestimmt werden, wie der Browser mit diesem Problem umgehen soll. Zur Auswahl stehen die folgenden Werte:

- **visible:** Der Inhalt wird auf jeden Fall angezeigt und ragt über das Element hinaus.
- **hidden:** Überlaufender Inhalt wird an den Elementgrenzen abgeschnitten.
- **clip:** Überlaufender Inhalt wird wie bei **hidden** abgeschnitten, allerdings lässt sich diese Grenze mit der Eigenschaft `overflow-clip-margin` selbst festlegen.
- **scroll:** Der Browser zeigt Scroll-Leisten am Element an, um den Rest des Inhalts navigierbar zu machen.
- **auto:** Der Browser entscheidet selbst, wie er vorgeht.

Der Standardwert ist **visible**.

Abbildung 4.4 zeigt die fünf Werte in genau dieser Reihenfolge. Verwendet wurde folgender CSS-Code:

```
1 div {
2   width: 100px;
3   height: 100px;
4   float: left; /* nebeneinander platzieren */
5 }
```

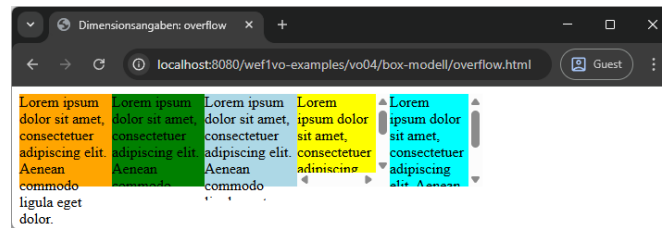


Abbildung 4.4: Bei `visible` ragt der Text aus der orangen Box heraus. In der grünen Box wird der Text durch `hidden` nach dem letzten Wort, das Platz findet, abgeschnitten. `clip` schneidet den Überlauf in der hellblauen Box ab, durch `overflow-clip-margin` geschieht dies aber erst nach 15px. `scroll` erzeugt in der gelben Box permanente Scroll-Leisten (auch horizontal, wo sie eigentlich nicht nötig wären). `auto` agiert am intelligentesten und platziert nur eine vertikale Scroll-Leiste.

```

6 #one {
7   overflow: visible;
8   background-color: orange;
9 }
10 #two {
11   overflow: hidden;
12   background-color: green;
13 }
14 #three {
15   overflow: clip;
16   overflow-clip-margin: 15px;
17   background-color: lightblue;
18 }
19 #four {
20   overflow: scroll;
21   background-color: yellow;
22 }
23 #five {
24   overflow: auto;
25   background-color: cyan;
26 }

```

4.1.3 Innenabstand

Mit dem *Innenabstand* wird der Leerraum zwischen dem Inhalt eines Elements und seinem Rand (oder Rahmen, falls vorhanden) definiert. Abbildung 4.5 zeigt schematisch die Position des Innenabstands in einem Element. Die Angabe ist für alle vier Seiten eines Elements einzeln, oder über eine zusammengefasste Shorthand-Syntax möglich.

Einzelne Angaben

Folgende Einzelangaben für den Innenabstand sind möglich:

- `padding-top`: Innenabstand nach oben.
- `padding-bottom`: Innenabstand nach unten.
- `padding-left`: Innenabstand nach links.

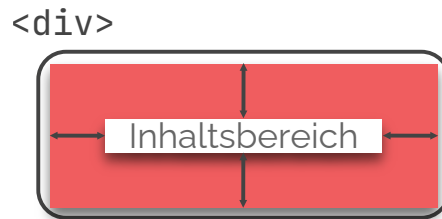


Abbildung 4.5: Der Innenabstand ist der Teil vom Inhalt zum Rand des Elements (z. B. <div>). Das Element wird dadurch größer.

- **padding-right:** Innenabstand nach rechts.

Im folgenden Code wird der Innenabstand eines <div>-Elements oben und unten explizit gesetzt.

```
1 div {
2   padding-top: 20px;
3   padding-bottom: 50px;
4 }
```

Shorthand-Syntax

Die Eigenschaft **padding** erlaubt die Angabe des *Innenabstands* für mehrere Seiten. Möglich sind eine bis vier numerische Angaben:

- Eine Angabe: Alle vier Seiten erhalten den gleichen Innenabstand.
- Zwei Angaben: Oben und unten (1. Wert) sowie links und rechts (2. Wert) erhalten den gleichen Innenabstand.
- Drei Angaben: Der Innenabstand wird für oben (1. Wert), rechts und links (2. Wert) und unten (3. Wert) gesetzt.
- Vier Angaben: Der Innenabstand wird für oben (1. Wert), rechts (2. Wert), unten (3. Wert) und links (4. Wert) gesetzt.

Im folgenden Beispiel werden alle vier Seiten mit einer Anweisung gesetzt.

```
1 div {
2   padding: 20px 15px 25px 10px;
3 }
```

4.1.4 Rahmen

Ein *Rahmen* wird immer am Rand eines Elements platziert. Abbildung 4.6 zeigt schematisch die Position. Die Angaben zum Rahmen (Rahmenstärke, Rahmenstil und Rahmenfarbe) können einzeln oder mit einer Shorthand-Syntax gemacht werden.

Einzelne Angaben

Selbst bei den Einzelangaben sind wieder mehrere Möglichkeiten vorhanden. So können die Angaben für Rahmenstärke, -stil und -farbe für alle vier Seiten gemeinsam oder auch einzeln gemacht werden.



Abbildung 4.6: Der Rahmen liegt am Rand des Elements und vergrößert es dadurch um seine Breite.

Rahmenstärke: Die *Stärke* (auch Dicke) des Rahmens wird mittels `border-width` angegeben. Es folgt eine numerische Angabe als Wert.

Folgendes Codestück setzt die Rahmenstärke eines `<div>`-Elements auf 2px:

```
1 div {  
2   border-width: 2px;  
3 }
```

Sollen die einzelnen Seiten unterschiedlich gestaltet werden, kann auf die vier verschiedenen Einzelangaben `border-top-width` (für oben), `border-right-width` (für rechts), `border-bottom-width` (für unten) und `border-left-width` (für links) zurückgegriffen werden.

Rahmenstil: Die *Art* des angezeigten Rahmens wird durch `border-style` definiert. Als Werte stehen eine Reihe von Angaben zur Verfügung: `none` (kein Rahmen), `hidden` (ebenfalls kein Rahmen, nur bei Tabellen mit Eigenschaft `border-collapse: collapse` sinnvoll), `dotted` (gepunktet), `dashed` (gestrichelt), `solid` (durchgezogen) sowie `double` (doppelt durchgezogen). Die Angaben `groove`, `ridge`, `inset` und `outset` sind ebenfalls möglich und erzeugen verschiedene 3D-Rahmeneffekte.

Folgender Code definiert einen durchgezogenen Rahmen für alle vier Seiten:

```
1 div {  
2   border-style: solid;  
3 }
```

Um unterschiedlichen Seiten eines Elements unterschiedliche Rahmenstile zuzuweisen, existieren wiederum verschiedene Einzelangaben: `border-top-style` (für oben), `border-right-style` (für rechts), `border-bottom-style` (für unten) und schließlich `border-left-style` (für links).

Rahmenfarbe: Mit der Eigenschaft `border-color` kann die *Farbe* des Rahmens spezifiziert werden. Möglich sind wie immer die in Abschnitt 3.7.3 erläuterten Farbangaben.

Folgender Code gibt erzeugt einen roten Rahmen:

```
1 div {  
2   border-color: red;  
3 }
```

Auch hierbei ist es möglich, allen vier Seiten unterschiedliche Farben zu geben. Die Eigenschaften dazu lauten: `border-top-color` (für oben), `border-right-color` (für rechts), `border-bottom-color` (für unten) und `border-left-color` (für links).



Abbildung 4.7: Der Außenabstand wird vom Rand des Elements weg berechnet und gehört nicht mehr zum eigentlichen Element.

Shorthand-Syntax

Mittels der Eigenschaft **border** ist es möglich, die Angaben für Rahmenstärke, -stil und -farbe gemeinsam anzugeben. Die Werte werden dazu in dieser Reihenfolge durch Leerzeichen getrennt angegeben.

Folgender Code erzeugt einen 1px starken, durchgehenden schwarzen Rahmen:

```
1 div {
2   border: 1px solid black;
3 }
```

Wiederum ist es möglich, die Shorthand-Syntax nur für einzelne Seiten des Rahmens anzuwenden. Dazu können die Eigenschaften **border-top** (für oben), **border-right** (für rechts), **border-bottom** (für unten) sowie **border-left** (für links) verwendet werden. Die Angabe der Werte erfolgt in der gleichen Reihenfolge.

4.1.5 Außenabstand

Der *Außenabstand* ist der Abstand eines Elements von seinem Eltern- oder Nachbar-element. Im Gegensatz zu Innenabstand und Rahmen liegt der Außenabstand bereits außerhalb des Elements – Abbildung 4.7 stellt dies schematisch dar. Der Außenabstand kann auch negative Werte annehmen. In diesem Fall kommt es zur Überlappung von Elementen.

Wie beim Innenabstand ist die Angabe für alle vier Seiten eines Elements einzeln, oder über eine zusammengefasste Shorthand-Syntax möglich.

Einzelne Angaben

Folgende Einzelangaben für den Außenabstand sind möglich:

- **margin-top**: Außenabstand nach oben.
- **margin-bottom**: Außenabstand nach unten.
- **margin-left**: Außenabstand nach links.
- **margin-right**: Außenabstand nach rechts.

Im folgenden Code wird der Außenabstand eines **<div>**-Elements links und unten explizit gesetzt:

```
1 div {
2   margin-left: 10px;
3   margin-bottom: 20px;
4 }
```

Shorthand-Syntax

Die Eigenschaft `margin` erlaubt die Angabe des Außenabstands für mehrere Seiten (analog zu `padding`). Möglich sind eine bis vier numerische Angaben:

- Eine Angabe: Alle vier Seiten erhalten den gleichen Außenabstand.
- Zwei Angaben: Oben und unten (1. Wert) sowie links und rechts (2. Wert) erhalten den gleichen Außenabstand.
- Drei Angaben: Der Außenabstand wird für oben (1. Wert), rechts und links (2. Wert) und unten (3. Wert) gesetzt.
- Vier Angaben: Der Außenabstand wird für oben (1. Wert), rechts (2. Wert), unten (3. Wert) und links (4. Wert) gesetzt.

Im folgenden Beispiel werden alle vier Seiten mit einer Anweisung gesetzt.

```
1 div {  
2   margin: 10px 15px 30px 20px;  
3 }
```

4.1.6 Besonderheiten und Wechsel der Box-Eigenschaften

Im Umgang mit dem Box-Modell gibt es zwei Besonderheiten, die im Folgenden erklärt werden sollen: Margin Collapse und das alternative (IE) Box-Modell. Darüber hinaus kann mit der CSS-Eigenschaft `display` bestimmt werden, wie sich eine Box verhält.

Margin Collapse

Liegen zwei Blockelemente mit oberen und unteren Außenabständen übereinander, so *kollabieren* die angrenzenden Außenabstände und es bleibt lediglich der größere von beiden übrig. Dies ist der Grund, warum etwa bei aufeinanderfolgenden Absätzen nicht überdurchschnittlich viel Platz entsteht. Elemente die nebeneinander liegen (mit linken und rechten Außenabständen) sind von dieser Besonderheit nicht betroffen.

Ein Beispiel: Absatz 1 hat einen Außenabstand von 16px oben und unten. Absatz 2 liegt unterhalb und hat einen Außenabstand von 50px oben und unten. Der Abstand zwischen beiden Elementen ist nun nicht (wie vielleicht erwartet) 66px, sondern 50px. Die beiden Abstände kollabieren und der größere kommt zum Zug. Abbildung 4.8 zeigt dies.

Das alternative (IE) Box-Modell

Eine weitere Besonderheit ist das *alternative Box-Modell*, auch bekannt als *IE Box-Modell*. Internet Explorer bis Version 5.5 und auch nach wie vor im Quirks-Modus (Anzeigen einer Website, die keinen DOCTYPE besitzt) berechnet das Box-Modell leicht anders. Anstatt, wie vom W3C definiert, zur Breite des Elements Innenabstand und Rahmen dazu zu zählen, werden diese von der Breite subtrahiert.

Abbildung 4.9 stellt das Beispiel vom Anfang des Kapitels im alternativen Box-Modell dar. Wird die Breite des Elements mit 90px angegeben, der Innenabstand mit zwei Mal 10px und der Rahmen mit zwei Mal 5px, so werden die 30px aus Innenabstand und Rahmen nun von den 90px abgezogen, womit effektiv nur mehr 60px für den Inhalt

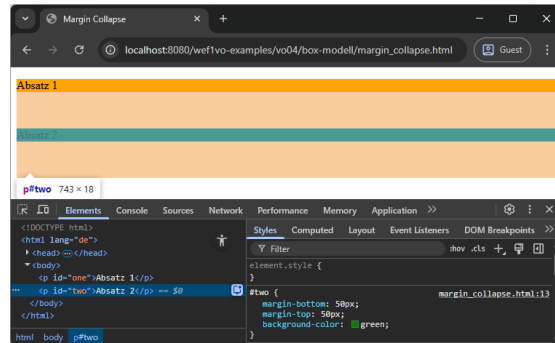


Abbildung 4.8: Der zweite Absatz hat einen oberen und unteren Außenabstand von 50px (ocker dargestellt). Der obere Abstand grenzt direkt an den ersten Absatz an – dies zeigt, dass der untere Abstand des ersten Absatzes (16px) „verschluckt“ wurde.

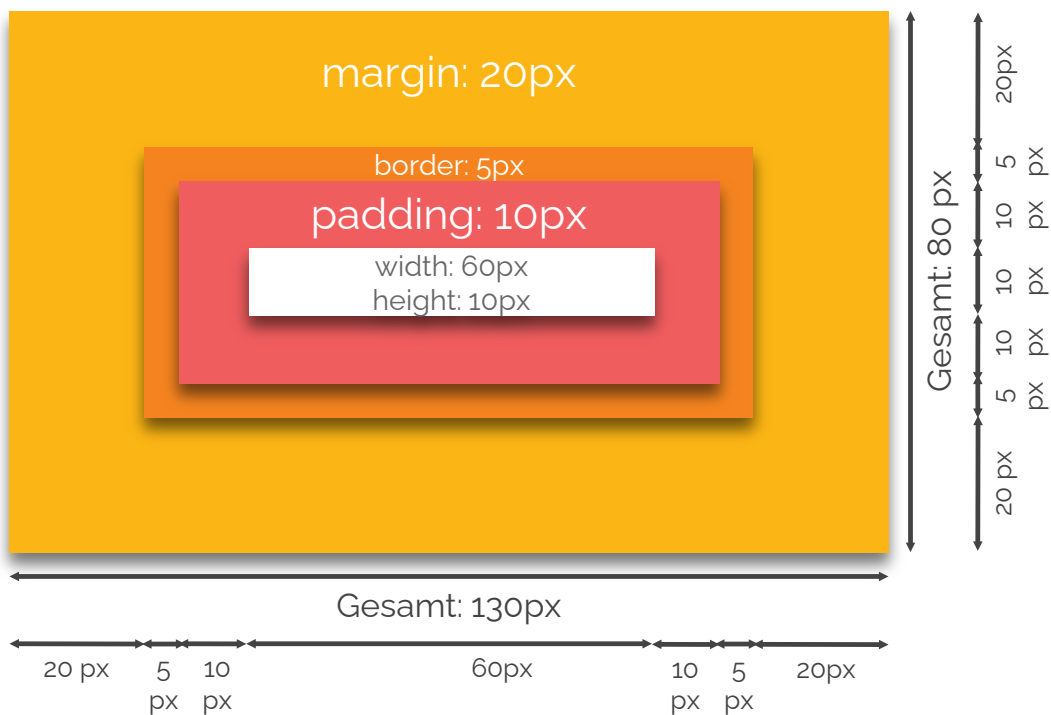


Abbildung 4.9: Im alternativen (IE) Box-Modell sind in der Breite von 90px Innenabstand und Rahmen bereits mit eingerechnet, daher ist die Gesamtbreite nur 130px. Die Gesamthöhe beträgt ebenso nur 80px.

übrig bleiben. Danach wird noch der Rahmen mit zwei Mal 20px addiert, was eine Gesamtbreite von 130px (anstatt der 160px im Standard Box-Modell) ergibt. Ebenso bleiben aus diesem Grund bei einer Höhe von 40px effektiv nur noch 10px übrig, die Gesamthöhe beträgt 80px (statt 110px).

Diese Art der Breitenberechnung erscheint – obwohl sie ursprünglich als „IE Box Model Bug“ bekannt war – nicht völlig abwegig. Innenabstand und Rahmen liegen

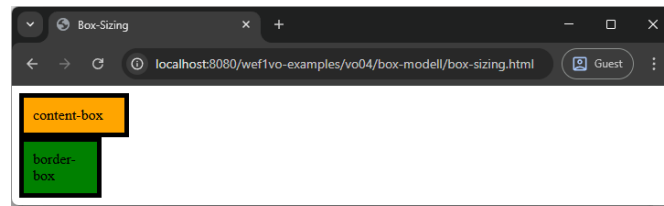


Abbildung 4.10: `div`-Element 1 wird mit dem Wert `content-box`, also nach dem Standard W3C Box-Modell gerendert. `div`-Element 2 wird mit `border-box` dargestellt, also mit dem alternativen (IE) Box-Modell.

schließlich noch im Element, daher ist es nicht unverständlich, dass `width` nicht nur die Breite des Inhalts (wie beim Standard-Box-Modell) sondern die Breite des gesamten Elements (also Inhalt + Innenabstand + Rahmen) angibt.

Diesen Umstand hat auch das W3C erkannt und im CSS Box Sizing Module Level 3 [9] die Eigenschaft `box-sizing` definiert. Sie kann die Werte `content-box` oder `border-box` annehmen. Der erste Wert ist die Standardeinstellung und zieht zur Berechnung das normale W3C Box-Modell heran, der zweite den im Internet Explorer ursprünglich implementieren Modus, der die Breite bis einschließlich des Rahmens berechnet. Obwohl die Spezifikation erst ein Working Draft ist, wird sie von allen Browsern schon lange unterstützt (da sie zuvor in einer anderen Spezifikation definiert wurde).

Folgender Code stellt zwei `<div>`-Elemente mit 90px Breite, 10px Innenabstand und 5px breitem Rahmen in den zwei Box-Modell Varianten dar. Abbildung 4.10 zeigt die Auswirkungen.

```

1 div {
2   width: 90px;
3   padding: 10px;
4   border: 5px solid black;
5 }
6 #one {
7   box-sizing: content-box;
8   background-color: orange;
9 }
10 #two {
11   box-sizing: border-box;
12   background-color: green;
13 }
```

Wechsel der Box-Eigenschaften

Das Browser-Stylesheet gibt vor, welche Elemente einen eigenen Block erzeugen (z. B. `<p>` oder `<div>`) und welche auf Absatzebene (inline) wirken (z. B. `<em>` oder `<img>`). Letztere nehmen nur so viel Platz wie nötig ein, die Eigenschaften `width` und `height` greifen dabei nicht, ebenso sind nur horizontale `margin`-Angaben möglich.

Die CSS-Eigenschaft `display` erlaubt es jedoch, den Charakter eines jeden Elements zu verändern. Sie kann die folgenden Werte annehmen:

- **inline:** Stellt ein Element als Inline-Element, d. h. als Phrasing content dar.

- **block**: Macht ein Element zum Blockelement, alle beschriebenen Dimensionsangaben sind wirksam.
- **inline-block**: Erstellt eine Box, die zwar innerhalb einer Textzeile existiert, aber wie ein Blockelement formatierbar ist.
- **list-item**: Erzeugt zwei Boxen, eine Hauptbox die sich wie ein Blockelement verhält und eine zweite, die das Aufzählungszeichen der Liste enthält.
- **none**: Erzeugt keine Box, d. h. das Element wird nicht mehr dargestellt.

Darüber hinaus gibt es mittels **table**, **table-cell**, **tablecolumn**, **table-row**, sowie **table-column-group**, **table-row-group**, **tableheader-group** und schließlich noch **table-footer-group** noch die Möglichkeit, Elementen das Verhalten von Tabellen bzw. deren Elementen zu geben. Auch für verschiedene Layoutmöglichkeiten, die in CSS3-Modulen ergänzt wurden, wird die Eigenschaft **display** verwendet. Details dazu können MDN¹ entnommen werden bzw. folgen in Vorlesung 5.

4.2 Positionierung von Elementen

Die Anordnung von Elementen erfolgt im Browser im Normalfall im so genannten *Document Flow* (auch Normal Flow) genannt. Dieser ordnet Elemente in der Reihenfolge an, wie sie im Quellcode erscheinen. Blockelemente werden als Boxen, Inline-Elemente in der Zeile mit anderen Elementen dargestellt.

Soll diese automatische Positionierung von Elementen geändert werden, kann dies primär mit der Eigenschaft **position** [10] geschehen. Sie bietet fünf mögliche Positionierungsarten an, die im den folgenden Abschnitten erläutert werden. Zusätzlich dazu kann mit der Eigenschaft **float** ebenfalls eine Neupositionierung erreicht werden. Diese Methode wird in Abschnitt 4.2.6 beschrieben.

4.2.1 Statische Positionierung

Statische Positionierung wird durch das Setzen von **position: static** erreicht. Diese Angabe ist jedoch im Normalfall nicht nötig, da die statische Positionierung dem Document Flow entspricht. Dies wird nur benötigt, um anders positionierte Elemente wieder in den Document Flow zu bringen.

Folgender Code gibt explizit für drei `<div>`-Elemente eine statische Positionierung an. Das Ergebnis ist in Abbildung 4.11 ersichtlich.

```
1 div {  
2   border: 1px solid black;  
3   width: 250px;  
4   height: 100px;  
5   margin: 10px;  
6   position: static;  
7 }  
8 #one {  
9   background-color: orange;  
10 }  
11 #two {  
12   background-color: green;
```

¹<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/display>



Abbildung 4.11: Die drei `<div>`-Elemente sind normal statisch positioniert, genauso als ob die Angabe von `position: static` nicht vorhanden wäre.

```
13 }
14 #three {
15   background-color: yellow;
16 }
```

4.2.2 Relative Positionierung

Relative Positionierung erfolgt durch die Angabe von `position: relative`. Das Element wird dadurch relativ zu seinem normalen Auftreten im Document Flow positioniert, es wird aber nicht aus dem Document Flow entfernt. Dies ist daran ersichtlich, dass die nachfolgenden Elemente nicht nachrücken, also immer noch so platziert sind, als ob das Element an seiner ursprünglichen Stelle wäre.

Um aber eine Neupositionierung zu erreichen, reicht die Angabe von `position: relative` alleine nicht aus. Zusätzlich muss eine Positionsangabe gemacht werden. Diese kann mit Hilfe der Eigenschaften `top` (von oben), `right` (von rechts), `bottom` (von unten) oder `left` (von links) geschehen. Die Elemente arbeiten mit regulären numerischen Werten. Der Bezugspunkt ist die ursprüngliche Position im Document Flow.

Sollen eventuell entstehende Überlappungen behandelt werden, kann bei den betroffenen Elementen die Eigenschaft `z-index` angegeben werden. Sie akzeptiert numerische Werte von 0 bis n . Je höher der Wert, desto weiter vorne wird das Element angeordnet.

Im folgenden Code ist das zweite der drei `<div>`-Elemente relativ positioniert. Abbildung 4.12 zeigt das Ergebnis im Browser.

```
1 div {
2   border: 1px solid black;
3   width: 250px;
4   height: 100px;
5   margin: 10px;
6 }
7 #one {
8   background-color: orange;
9 }
10 #two {
11   position: relative;
```



Abbildung 4.12: Das zweite `<div>`-Element ist relativ zu seiner alten Position um 80px nach rechts (von links ausgehend) und 30px nach unten (von oben ausgehend) verschoben.

```

12 left: 80px;
13 top: 30px;
14 background-color: green;
15 }
16 #three {
17 background-color: yellow;
18 }

```

4.2.3 Absolute Positionierung

Absolute Positionierung wird durch Verwendung von `position: absolute` erreicht. Diese Angabe nimmt das Element aus dem Document Flow heraus, d.h. andere Elemente unterhalb rücken einfach nach. Wiederum geschieht die Positionierung durch Angaben von `top`, `right`, `bottom` oder `left`.

Der Bezugspunkt für eine Neupositionierung ist das nächste Elternelement, das *nicht* mit `position: static` positioniert wurde. Gibt es kein solches, dann ist das Browserfenster der Bezugspunkt. Mögliche Überlappungen können wiederum mit der Eigenschaft `z-index` verwaltet werden.

Im folgenden Beispiel ist das zweite Element absolut positioniert. Das dritte Element rückt direkt an das erste, da das zweite nicht mehr im Document Flow enthalten ist. Abbildung 4.13 zeigt dieses Verhalten.

```

1 div {
2   border: 1px solid black;
3   width: 250px;
4   height: 100px;
5   margin: 10px;
6 }
7 #one {
8   background-color: orange;
9 }
10 #two {
11   position: absolute;
12   left: 80px;
13   top: 30px;
14   background-color: green;

```

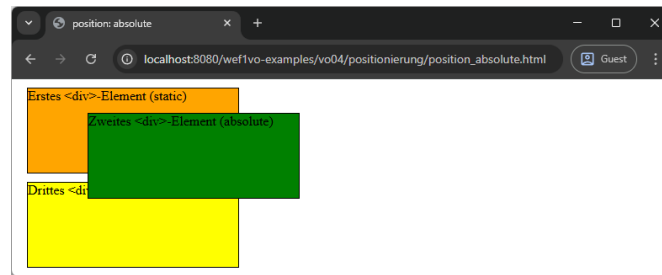


Abbildung 4.13: Das zweite `<div>`-Element ist absolut positioniert. Da kein anderes nicht-statisch platziertes Element als Elternelement vorhanden ist, wird es 80px vom linken und 30px vom oberen Browserfensterrand positioniert.

```

15 }
16 #three {
17   background-color: yellow;
18 }

```

4.2.4 Fixe Positionierung

Auch bei *fixer Positionierung* wird das Element aus dem Document Flow entfernt. Verantwortlich ist die Angabe von `position: fixed`. Im Unterschied zu absoluter Positionierung wird das Element im Bezug zum Ausgabemedium positioniert, bleibt also immer an derselben Stelle am Bildschirm oder am Papier, falls die Seite gedruckt wird. Dies kann etwa für eine mitscrollende Navigation verwendet werden, sollte aber überlegt eingesetzt werden, da der Effekt mitunter störend sein kann.

Auch hier erfolgt die Positionierung mit Hilfe von `top`, `right`, `bottom` oder `left`. Der Bezugspunkt ist der Viewport, also das Browserfenster. Überlappungen können mit `z-index` verwaltet werden.

Folgender Code positioniert das zweite Element fix. Muss gescrollt werden, so bleibt es immer an derselben Stelle. Abbildung 4.14 stellt dies annähernd dar – selbst ausprobieren wird ans Herz gelegt.

```

1 div {
2   border: 1px solid black;
3   width: 250px;
4   height: 100px;
5   margin: 10px;
6 }
7 #one {
8   background-color: orange;
9 }
10 #two {
11   position: fixed;
12   left: 80px;
13   top: 30px;
14   background-color: green;
15 }
16 #three {
17   background-color: yellow;
18 }

```

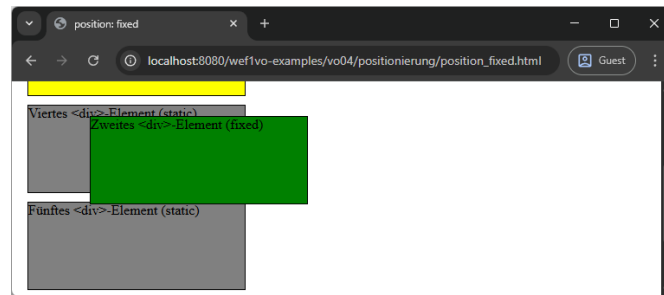



Abbildung 4.14: Das zweite `<div>`-Element ist fix positioniert. Obwohl im Browserfenster schon nach unten gescrollt wurde, befindet sich das Element nach wie vor 80px von links und 30px vom oberen Rand des Browserfensters entfernt.

```
19 #four, #five, #six {
20   background-color: grey;
21 }
```

4.2.5 Sticky Positionierung

Die *sticky Positionierung* stellt eine Hybridform aus relativer und fixer Positionierung dar und wird über `position: sticky` erreicht. Die Angaben `top`, `right`, `bottom` oder `left` geben dabei den Minimalabstand an, den ein Element von einer Seite des Viewports haben muss. Solange der Wert größer ist, verhält sich das Element wie ein relativ positioniertes, wird er unterschritten (etwa durch Scrollen), wird das Element fix bei diesem Abstand positioniert.

Im folgenden Beispiel wird ein Element sticky positioniert. Zunächst bleibt es regulär im Document Flow und ist um 80 Pixel eingerückt. Wird abwärts gescrollt und der obere Abstand des Elements zum Browserfensterrand unterschreitet 30 Pixel, bleibt das Element an dieser Stelle fix stehen. Dies entspricht Abbildung 4.14.

```
1 div {
2   border: 1px solid black;
3   width: 250px;
4   height: 100px;
5   margin: 10px;
6 }
7
8 #one {
9   background-color: orange;
10 }
11
12 #two {
13   position: sticky;
14   left: 80px;
15   top: 30px;
16   background-color: green;
17 }
18
19 #three {
20   background-color: yellow;
```

```
21 }
22
23 #four, #five, #six {
24   background-color: grey;
25 }
```

4.2.6 Floats

Die CSS-Eigenschaft `float` nimmt ein Element aus dem Document Flow und verschiebt es ganz nach links oder rechts, bis es beim nächsten übergeordneten Element ansteht. Eine Verschiebung nach links wird mit `float: left`, eine Verschiebung nach rechts mit `float: right` erwirkt. Nachfolgende Elemente rutschen an den freigewordenen Platz, der Text in diesen Elementen *umfließt* jedoch (im Unterschied etwa zur absoluten Positionierung) das gefloatete Element.

Ursprünglich entworfen wurde die Eigenschaft `float`, um ein Bild innerhalb eines Absatzes einfach von Text umfließen lassen zu können. Die Eigenschaft kann jedoch auch zum schnellen horizontalen Anordnen von Elementen oder sogar für komplette Layouts (siehe Abschnitt 4.3) verwendet werden.

Soll ein Nachrücken der nachfolgenden Elemente verhindert werden, so kann die Eigenschaft `clear` mit den Werten `left`, `right` oder `both` (je nachdem, in welche Richtung zuvor gefloatet wurde) beim nachfolgenden Element angegeben werden.

Abbildung 4.15 zeigt vier verschiedene Anwendungsfälle von `float` und `clear`. Der Code zum Beispiel in Abbildung 4.15 (d) lautet wie folgt:

```
1 div {
2   border: 1px solid black;
3   margin: 10px;
4 }
5 #one {
6   background-color: orange;
7   float: right;
8   width: 250px;
9   height: 100px;
10 }
11 #two {
12   background-color: green;
13   float: left;
14   width: 300px;
15   height: 200px;
16 }
17 #three {
18   background-color: yellow;
19   clear: left;
20   width: 400px;
21   height: 100px;
22 }
```

Floats haben noch einige weitere Besonderheiten, die oftmals bei der Arbeit auftreten. Dazu gehören etwas das Kollabieren übergeordneter Elemente, wenn die darin enthaltenen alle gefloatet sind oder auch die Thematik des Float-Clearings, also Techniken, wie effektiv die `clear`-Eigenschaft angewandt werden kann. Diese Dinge sind ausführlich und gut illustriert in [1] beschrieben.



Abbildung 4.15: Mehrere Anwendungsfälle von `float` und `clear`. Das erste Element ist nach rechts gefloatet, das zweite und dritte rücken auf (a), das erste Element ist nach rechts, das zweite nach links gefloatet, das dritte rückt auf und der Text umfließt das Element (b), die ersten beiden Elemente sind rechts bzw. links gefloatet, das dritte erhält eine `clear: right` Anweisung und wird unter dem rechten positioniert (c), die ersten beiden Elemente sind links und rechts gefloatet, das dritte wird durch `clear: left` unter dem linken positioniert (d).

4.3 Einfache Anordnungen und Layouts mit float

Floats werden (neben ihrer ursprünglichen Anwendung, Bilder von Text umfließen zu lassen) meist für zwei Dinge eingesetzt: das horizontale Anordnen von Elementen in kleinen Bereichen oder für größere Layouts. Angemerkt sei, dass diese Techniken zwar immer noch gültig sind, mittlerweile aber mit Hilfe von moderneren Tools wie Flexbox oder dem Grid Layout umgesetzt werden sollten. Mehr dazu in Vorlesung 5.

4.3.1 Horizontales Anordnen von Elementen

Um Elemente nebeneinander anordnen zu können, reicht eine `float:left` (bzw. eine `float:right`) Anweisung auf einem der Elemente. Dieses wird dann ganz nach links (bzw. rechts) verschoben, das zweite Element rutscht daneben – immer vorausgesetzt, dass genug Platz ist. Ändert sich die Größe des gefloateten Elements, reagiert der daneben platzierte Inhalt darauf und rückt entsprechend nach.

Ein Beispiel ist eine so genannte Staff-Card, die sich auf vielen Firmenwebseiten findet. Hierbei wird neben das Foto einer Person deren Name und Position in der Firma platziert. Das Foto floatet links, die Daten werden entsprechend daneben platziert. Zunächst das HTML-Markup:

```
1 <section class="staff-card">
2   <div class="staff-photo">
```

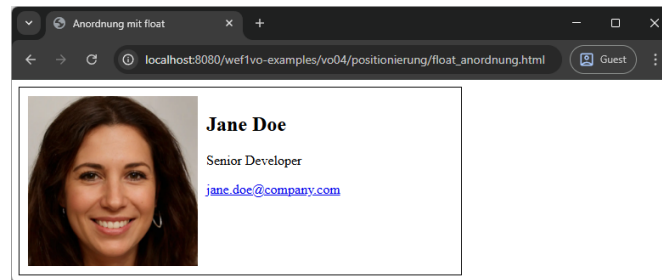


Abbildung 4.16: Das `<div>`-Element mit dem Bild floatet links, der Text wird rechts daneben angeordnet.

```

3      <!-- Image generated by https://thispersondoesnotexist.com/ -->
4      
5    </div>
6    <div class="staff-details">
7      <h2>Jane Doe</h2>
8      <p>Senior Developer</p>
9      <p><a href="mailto:jane.doe@company.com">jane.doe@company.com</a></p>
10   </div>
11 </section>

```

Dieses wird mit folgendem CSS-Code formatiert. Das Ergebnis ist in Abbildung 4.16 ersichtlich.

```

1 .staff-card {
2   width: 500px;
3   height: 200px;
4   padding: 10px;
5   border: 1px solid black;
6 }
7
8 .staff-photo {
9   height: 100%;
10  float: left;
11  margin-right: 10px;
12 }
13
14 .staff-photo img {
15   height: 100%;
16 }

```

4.3.2 Einfache Layouts

Nach demselben Prinzip können Floats verwendet werden, um einfache Layouts zu erstellen. Im folgenden Beispiel wird ein dreispaltiges Layout erstellt, das nicht die gesamte Breite der Seite in Anspruch nimmt, also sich in einem zentrierten Container befindet. Das Layout besteht aus Kopf- und Fußzeile. Dazwischen befinden sich die Navigation, der Bereich für den Hauptinhalt und rechts eine Sidebar. Abbildung 4.17 stellt die grundsätzliche Aufteilung dar.

Das HTML-Gerüst für das Layout bedient sich der bereits bekannten Strukturelemente. Der Kopfbereich verwendet erwartungsgemäß das `<header>`-Element, für die

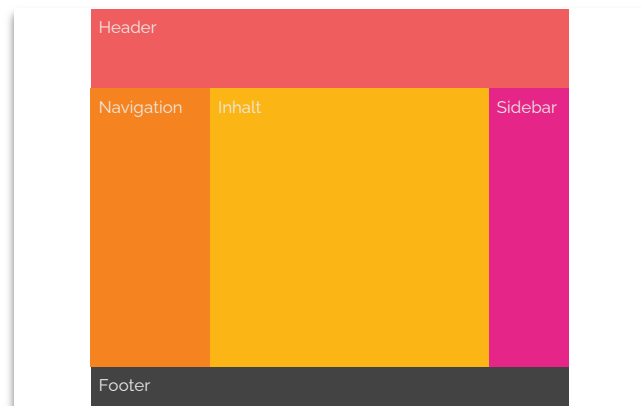


Abbildung 4.17: Die Seite wird in Header, Navigation, Inhalt, Sidebar und Footer aufgeteilt. Navigation, Inhalt und Sidebar sollen dabei nebeneinander positioniert sein.

Navigation kommt `<nav>` zum Einsatz. Der Inhaltsbereich wird mit `<main>` ausgezeichnet, darin wird eine `<section>` verwendet, um den eigentlichen Inhalt zu platzieren und `<aside>` für die Sidebar. Für den Fußbereich kommt `<footer>` zum Einsatz. Um alle Elemente herum wird mit einem `<div>`-Element ein unsichtbarer Container platziert, der zentriert wird und eine maximale Breite bekommt, damit der Inhalt nicht die komplette Browserbreite einnimmt. Dies ist vor allem bei hochauflösenden oder ultrabreiten Displays von Vorteil.

```

1 <body>
2   <div id="container">
3     <header>
4       <h1>Überschrift</h1>
5     </header>
6     <nav>
7       <ul>
8         <li>Navigation 1</li>
9         <li>Navigation 2</li>
10      </ul>
11    </nav>
12    <main>
13      <section id="content">
14        <p>Lorem ipsum dolor sit amet, [...]</p>
15      </section>
16      <aside>
17        <p>Lorem ipsum dolor sit amet, [...]</p>
18      </aside>
19    </main>
20    <footer>
21      <p>&copy; 2025 Der*Die Autor*in</p>
22    </footer>
23  </div>
24 </body>

```

Die Anordnung erfolgt nun komplett über CSS. Zunächst werden mit dem Universal-selektor alle Außenabstände auf 0 gesetzt und das Box-Modell auf `border-box` gesetzt.

Damit können später gefahrlos `padding`- und `border`-Angaben gesetzt werden, ohne dass die nebeneinanderliegenden Spalten die berechnete Gesamtbreite übersteigen, und somit nicht mehr nebeneinander Platz hätten. Der äußere Container wird auf eine fixe Breite gesetzt und wird mit Hilfe einer Außenrandangabe, bei der die vertikalen Werte auf 0, die horizontalen auf `auto` gesetzt werden, zentriert. Der Container bekommt eine Hintergrundfarbe, ebenso der Header.

Danach werden Navigation und Hauptbereich gefloatet – links und rechts, die Summe ergibt genau 100 % der Containerbreite. Die Navigation, meist kürzer als der Inhalt, erhält keine Hintergrundfarbe. Hier „scheint“ die Farbe des Containers durch. Der Hauptbereich bekommt eine Hintergrundfarbe, die für die Sidebar verwendet wird. Diese wird rechts gefloatet, der Abschnitt für den Inhalt links. Die Gesamtbreite entspricht der Breite des Hauptbereichs. Der Abschnitt für den Inhalt wiederum benötigt zur Unterscheidung eine eigene Hintergrundfarbe.

Schließlich soll der Footer unter die beiden Spalten gesetzt werden. Dafür wird `clear: both` verwendet. Der Footer bekommt ebenfalls eine Hintergrundfarbe. Alle fünf Bereichselemente bekommen schließlich noch etwas `padding` gesetzt, damit der Text nicht direkt am Rand ihrer Elemente „klebt“.

Das fertige Layout ist in Abbildung 4.18 ersichtlich.

```
1 * {  
2   margin: 0;  
3   box-sizing: border-box;  
4 }  
5 body {  
6   color: rgb(236 233 230);  
7   font-family: Raleway, Arial, sans-serif;  
8   line-height: 2em;  
9 }  
10 div#container {  
11   width: 960px;  
12   margin: 0 auto;  
13   background-color: rgb(255 132 0);  
14 }  
15 header {  
16   background-color: rgb(250 94 94);  
17 }  
18 nav {  
19   width: 240px;  
20   float: left;  
21 }  
22 main {  
23   width: 720px;  
24   float: right;  
25   background-color: rgb(228 42 135);  
26 }  
27 section#content {  
28   width: 600px;  
29   float: left;  
30   background-color: rgb(255 182 0);  
31   color: rgb(67 67 67);  
32 }  
33 aside {  
34   width: 120px;
```

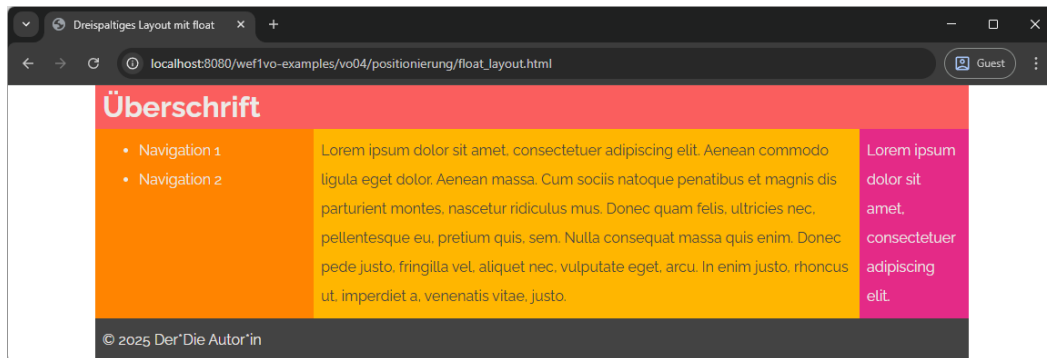


Abbildung 4.18: Das fertige Layout. Die nebeneinander angeordneten Bereiche werden durch `float`-Anweisungen erreicht.

```

35 float: right;
36 }
37 footer {
38   clear: both;
39   background-color: rgb(67 67 67);
40 }
41 header, nav, footer, section, aside {
42   padding: 0.5em;
43 }

```

4.4 Media Queries

In Zeiten, in denen mobile Endgeräte immer verstärkter zur Darstellung von Websites verwendet werden, auf der anderen Seite aber Desktop-Computer immer größere Auflösungen erreichen, wird es mit herkömmlichen Mitteln schwierig, Layouts möglichst gut dem Endgerät anzupassen. *Media Queries* [11–13] versuchen, dieses Problem auf effektive Art und Weise zu lösen.

4.4.1 Medienspezifische Stylesheets

Medienabhängige Stylesheets (ein Stylesheet für den Bildschirm und eines für den Druck etwa) gibt es schon in CSS 2.1 bzw. HTML 4.01. So kann bei der Einbindung mittels `<link>`-Tag das Attribut `media` angegeben werden, für welches Medium ein Stylesheet verwendet werden soll. In folgendem HTML-Fragment aus dem `<head>` eines Dokuments, wird für die Ausgabe am Bildschirm die Datei `style.css` und beim Druck die Datei `print.css` eingebunden:

```

1 <link rel="stylesheet" href="style.css" media="screen">
2 <link rel="stylesheet" href="print.css" media="print">

```

Auch bei der Einbindung mittels `@import` kann der Medientyp angegeben werden:

```

1 @import url("style.css") screen;
2 @import url("print.css") print;

```

Tabelle 4.1: Die derzeit verfügbaren (und sinnvoll interpretierten) Medientypen.

Medientyp	Ausgabegerät
all	Alle Ausgabegeräte.
print	Drucker und Geräte, die einen Drucker simulieren (z. B. die Druckvorschau im Browser).
screen	Alle bildschirmorientierten Geräte, die nicht unter print fallen. Dazu zählen auch Screenreader.

Medienspezifische Angaben können aber auch innerhalb eines Stylesheets gemacht werden. Dabei werden die entsprechenden Angaben mit der `@media`-Anweisung gruppiert. Im Folgenden etwa wird nur beim Ausdruck die Navigation ausgeblendet, andere Regeln im Stylesheet bleiben davon unbeeinflusst und sind auch beim Druck wirksam:

```

1 @media print {
2   nav {
3     display: none;
4   }
5 }
```

4.4.2 Medientypen

Es gibt eine Reihe von verschiedenen Schlüsselwörtern, die im Zusammenhang mit `@media`-Angaben verwendet werden können. Diese haben sich über die Zeit jedoch als oft nicht mehr passend herausgestellt, sodass in der letzten Überarbeitung der Media Query Spezifikation [13, Abschnitt 2.3] nur noch drei Typen empfohlen werden, die von Browsern erkannt und unterschiedlich gehandhabt werden sollen. Tabelle 4.1 stellt diese dar.

Als veraltet angesehen werden die Typen **tty** (textbasierte Computerterminals), **tv** (TV-Geräte), **projection** (Projektoren), **handheld** (mobile Geräte mit geringer Auflösung), **braille** (Braille-Zeilen-basierte Ausgabegeräte), **embossed** (Braille-Drucker), **aural** (Sprachsynthesizer) und **speech** (Screenreader). Während einige dieser Typen zunächst durchaus sinnvoll erscheinen (etwa **handheld** für mobile Geräte), hat sich gezeigt, dass eine Unterscheidung durch die technologische Entwicklung (Smartphones mit sehr hoher Auflösung) schwierig wurde. Stattdessen wurden die Medientypen reduziert und feinere Unterscheidungen können mit Hilfe der diversen Medienmerkmale (siehe Abschnitt 4.4.4) getroffen werden. Es wird daher davon ausgegangen, dass Medientypen in Zukunft ganz verschwinden und Unterscheidungen rein anhand der Medienmerkmale getroffen werden.

4.4.3 Aufbau von Media Queries

Mit der CSS3 Media Query Spezifikation [11] und nachfolgenden Media Queries Level 4 und 5 Standards [12, 13] wurde auf dem Modell der medienspezifischen Stylesheets aufgebaut und dieses um Medienmerkmale erweitert. Formal besteht eine Media Query aus einem optionalen Media Query Modifikator (**only** oder **not**), einem optionalen Medientyp (siehe Abschnitt 4.4.2) und 0 oder mehr Medienmerkmalen (siehe Abschnitt

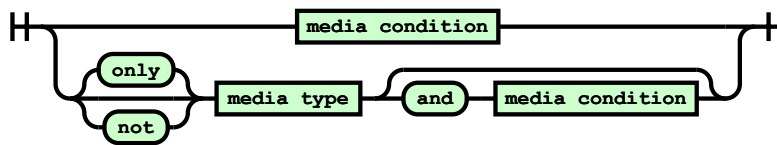


Abbildung 4.19: Der schematische Aufbau einer Media Query. Quelle: [13].

4.4.4). Die Kombination von mehreren Medienmerkmalen mit Hilfe boolescher Algebra wird dann allgemeiner als Medienbedingung (engl. „media condition“) bezeichnet.

Eine Media Query ist dabei ein logischer Ausdruck, der entweder **true** oder **false** ergibt. Ersteres ist dabei der Fall, wenn der optional spezifizierte Medientyp mit dem gerade verwendeten Gerät übereinstimmt und die Medienbedingung (mit den darin enthaltenen Medienmerkmalen) erfüllt ist. Abbildung 4.19 stellt dies schematisch dar.

Im folgenden Beispiel wird etwa eine Media Query für die Ausgabe am Bildschirm und Geräte mit einer maximalen Viewportbreite von 480 Pixeln erstellt. In den geschweiften Klammern stehen dann die CSS-Regeln für diesen Fall:

```
1 @media screen and (max-width: 480px) {
2   /* CSS-Regeln */
3 }
```

Wird vor die Media Query das Schlüsselwort **not** gestellt, dann wird sie entsprechend negiert, würde also für alle Anwendungsfälle *außer* dem spezifizierten gelten.

Durch die Angabe des Schlüsselworts **only** vor der Media Query kann die gesamte Media Query vor alten Browsern, die dieses Feature nicht unterstützen, versteckt werden. Ein alter Browser würde nämlich zwar den ersten Teil, d. h. die Angabe des Medientyps interpretieren, jedoch nicht die nachfolgenden Medienmerkmale. Dies könnte zur Folge haben, dass Styles angewandt werden, die eigentlich nicht gewünscht sind. Durch Voranstellen von **only** ignorieren alte Browser eine Media Query komplett.

4.4.4 Medienmerkmale

Medienmerkmale sind feiner aufgelöste Testkriterien als Medientypen. Sie erlauben es eine einzelne, spezifische Eigenschaft eines Geräts oder auch eines Browsers abzufragen. Syntaktisch sind sie zunächst wie CSS-Eigenschaften aufgebaut, sie bestehen aus dem Merkmal, einem Doppelpunkt und einem Wert auf den getestet wird, z. B. **max-width: 480px**, um auf eine maximale Breite von 480 Pixeln abzufragen. Allerdings werden Medienmerkmale immer in Klammern gesetzt und können in der neuesten Spezifikation auch Vergleichsoperatoren anstelle des Doppelpunkts haben oder einfach nur auf die Eigenschaft prüfen. So ist **(width <= 480px)** möglich oder auch **(color)**. Letzteres überprüft die Farbtiefe eines Ausgabegeräts und ist dann **true**, wenn der gelieferte Wert nicht 0 ist.

Die folgenden Medienmerkmale sind unter anderem verfügbar, viele besitzen auch noch die jeweiligen **min-** und **max-** Angaben. Diese sind meist zu bevorzugen, da etwas die Größe des Viewports eines Desktop-Browser selten der genauer Pixelwert ist und somit eine Abfrage auf Minimal- oder Maximalwerte wesentlich zuverlässiger ist. Eine

vollständige Auflistung findet sich in [11–13].

- **width**: Breite des Viewports (der innere Bereich des Browserfensters oder Ausgabegeräts, ohne Menüs, Scrollbalken etc.).
 - **min-width**: Die minimale Breite des Viewports (alles größer gleich des angegebenen Werts ergibt **true**).
 - **max-width**: Die maximale Breite des Viewports (alles bis zum angegebenen Wert ergibt **true**).
- **height**: Höhe des Viewports.
 - **min-height**: Minimale Höhe des Viewports.
 - **max-height**: Maximale Höhe des Viewports.
- **orientation**: Ausrichtung mit den Werten **portrait** und **landscape**. Interessant für mobile Endgeräte.
- **aspect-ratio**: Verhältnis zwischen **width** und **height**. Werte werden mit Schrägstrich notiert (z. B. 16/9).
 - **min-aspect-ratio**: Das minimale Seitenverhältnis.
 - **max-aspect-ratio**: Das maximale Seitenverhältnis.
- **color**: Die Farbtiefe des aktuellen Geräts in Bits pro Farbkomponente.
 - **min-color**: Die minimale Farbtiefe.
 - **max-color**: Die maximale Farbtiefe.
- **resolution**: Die Auflösung des Geräts in DPI (z. B. 72dpi), DPPX (Dots per Pixel) oder DPCM (Dots per CM).
 - **min-resolution**: Die minimale Auflösung.
 - **max-resolution**: Die maximale Auflösung.
- **pointer**: Erkennt das Vorhandensein eines bestimmten Eingabegeräts. Es existieren die Werte **none** (nur Tastatur), **coarse** (limitierte Genauigkeit wie Touchinput) und **fine** (präzise Genauigkeit wie etwa Mäuse, Touchpads oder Stifte).
- **prefers-reduced-motion**: Erkennt, ob die Systemeinstellungen von Nutzer*innen auf reduzierte Bewegungen gesetzt sind. Damit können etwa Animationen auf der Webseite deaktiviert werden. Werte sind **no-preference** oder **reduce**.
- **prefers-contrast**: Erkennt, ob Nutzer*innen erhöhten oder verringerten Kontrast bevorzugen. Werte sind **no-preference**, **high**, **low** oder **forced**.

4.4.5 Verknüpfung von Medienmerkmalen

In einer Media Query können mehrere Medienmerkmale miteinander verknüpft werden (was dann als Medienbedingung bezeichnet wird). Dies kann mit boolescher Algebra geschehen:

- Logisches Und: Mit dem **and** Schlüsselwort.
- Logisches Oder: Mit einem Komma (,) oder seit Media Queries Level 4 mit dem **or** Schlüsselwort.
- Logische Verneinung: Mit dem **not** Schlüsselwort.

Darüber hinaus kann mit Klammerung die Auswertung beeinflusst werden.

Die folgende Media Query gilt nur für Geräte, deren Breite zwischen 768 und 960 Pixeln liegt:

```
1 @media screen and (min-width: 768px) and (max-width: 960px) {  
2   /* CSS-Regeln */  
3 }
```

4.4.6 Viewport angeben

Damit eine Media Query auf einem mobilen Gerät sinnvoll verwendet werden kann, ist die Angabe des Viewports notwendig. Browser auf mobilen Geräten geben sich in der Regel nämlich wie Desktopbrowser aus, d. h. sie weisen wesentlich mehr Höhe und Breite aus, als ihnen eigentlich zur Verfügung steht, damit Webseiten, die nicht für mobile Ansicht optimiert wurden komplett (aber in der Regel sehr klein) dargestellt werden. Wäre das nicht der Fall, wären nicht optimierte Webseiten nur zum Teil sichtbar und es müsste auch horizontal gescrollt werden. Smartphones haben zwar Bildschirme mit vielen Pixeln, durch die hohe Pixeldichte (Device Pixel Ratio, DPR) müssen die horizontale und vertikale Auflösung durch die DPR dividiert werden, um die tatsächlichen „CSS-Pixel“ zu erhalten. So hat ein Smartphone mit 1.355 Pixel breitem Display und einer DPR von 3 eine Breite von 448 Pixeln.

Dieses Verhalten führt also dazu, dass Media Queries nur bedingt greifen, da der Browser am Mobiltelefon zunächst wesentlich mehr Breite als etwa die in der Media Query angegebenen 480 ausweist. Dies bedeutet, dass am Mobiltelefon trotzdem die Desktop-Variante angezeigt wird. Um dies zu verhindern, muss eine weitere Meta-Angabe in den `<head>` des Dokuments gesetzt werden:

```
1 <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Dadurch wird auf mobilen Geräten die Breite auf die tatsächliche Gerätebreite gelegt und der Zoom auf den Faktor 1 gestellt. Somit identifiziert sich obiges Smartphone mit 448 Pixeln und die Media Query für die Mobilansicht greift.

Mit der Angabe `user-scalable=no` könnte das Zoomen ganz verhindert werden (in der Regel nicht empfehlenswert), mit `minimum-scale` und `maximum-scale` kann die Zoomstufe begrenzt werden.

4.4.7 Media Queries einsetzen

Bei der Arbeit mit Media Queries geht es darum zu entscheiden, welches Layout auf Geräten akzeptabel funktioniert bzw. gut aussieht. Es müssen so genannte *Breakpoints* gesetzt werden, bei denen eine Layoutänderung stattfindet, d. h. wo etwa die Navigation nicht mehr links und der Inhalt rechts angeordnet werden, sondern beides untereinander. Ein Breakpoint wird durch eine `@media`-Regel definiert, mit der neue Style-Regeln ab oder bis zu einer gewissen Breite in Kraft treten.

Diese Breakpoints können einerseits an den Auflösungen bekannter Geräte² ausgerichtet werden. Allerdings kommen laufend neue Geräte mit immer neuen Auflösungen auf den Markt, was dieses Vorhaben mitunter schwierig macht.

²Eine Übersicht bietet etwa [3].

Ein aktuellerer Ansatz ist es, die Breakpoints am Inhalt und dem Design der eigenen Seite auszuwählen. Also selbst, nach dem eigenen ästhetischen Empfinden zu bestimmen, ab welcher Größe eine Änderung stattfinden soll. Sie können dazu nach zwei Ansätzen vorgehen: *Desktop first* und *Mobile first*.

Desktop first

Beim Desktop first Ansatz, wird zunächst das komplette Layout, optimiert für eine Desktop Auflösung definiert. Also etwa wie im zweispaltigen, seitenbreiten Layout mit zwei Spalten nebeneinander. Nun wird die Seite in immer schmäleren Breiten (etwa durch einfaches Zusammenschieben des Browsers oder mit Hilfe der des Device Modus³ der Chrome DevTools betrachtet (andere Browser bieten ebenfalls derartige Funktionalität an). Wenn das Layout nicht mehr gut aussieht (zu schmale Spalten etc.) oder überhaupt unschön umbricht, wird ein Breakpoint definiert, d. h. eine Media Query definiert, die ab dieser Breite etwas am Layout ändert. Dies kann natürlich an mehreren Stellen geschehen und sollte mit mehreren Geräten getestet werden.

Beim Desktop first Ansatz kommen in der Regel Medieneigenschaften wie `max-width` zum Zug, um alles unter einer bestimmten Auflösung abzuändern.

Im folgenden Beispiel kommt das dreispaltige Layout aus Abschnitt 4.3.2 zum Einsatz. Das HTML-Markup bleibt dabei völlig unverändert.

In der Desktop first Variante ist auch das CSS fast identisch, lediglich die Spaltenbreiten werden in Prozent anstatt Pixel angegeben. Dies ermöglicht eine Stufenlose Anpassung der Spalten, wenn etwas das Browserfenster verkleinert wird. Zunächst wird auch der Container fix mit 960 Pixeln breite angegeben.

Der CSS-Code für die wichtigsten Elemente (andere finden sich im Code zur Vorlesung):

```
1 div#container {
2   width: 960px;
3   margin: 0 auto;
4   background-color: rgb(255 132 0);
5 }
6 header {
7   background-color: rgb(250 94 94);
8 }
9 nav {
10  width: 25%;
11  float: left;
12 }
13 main {
14  width: 75%;
15  float: right;
16  background-color: rgb(228 42 135);
17 }
18 section#content {
19  width: 80%;
20  float: left;
21  background-color: rgb(255 182 0);
22  color: rgb(67 67 67);
23 }
```

³<https://developer.chrome.com/docs/devtools/device-mode/>

```
24 aside {  
25   width: 20%;  
26   float: right;  
27 }
```

Nun werden mit Hilfe von Media Queries mehrere Breakpoints gesetzt. Der erste passiert, wenn die Breite von 960 Pixeln unterschritten wird. Dabei wird die Breite des Containers auf 100 % umgestellt, sodass er sich fortan dem Viewport anpasst.

```
1 @media screen and (max-width: 960px) {  
2   div#container {  
3     width: 100%;  
4   }  
5 }
```

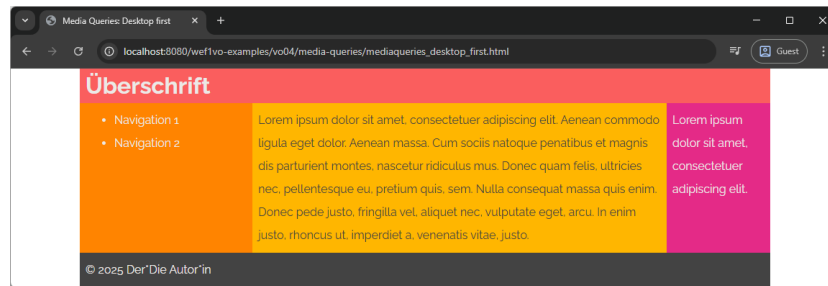
Der nächste Breakpoint wird bei 768 Pixeln gesetzt. Hier soll die Navigation über den zwei gefloateten Inhaltsbereichen platziert werden. Dazu wird das Floating aufgehoben und die Navigationspunkte nebeneinander angeordnet. Damit der Inhalt nun die volle Breite annimmt, wird das `<main>`-Element auf 100 % Breite gesetzt (das Floating nach rechts wird beibehalten, da das Element sonst leer wäre und kollabieren würde, dadurch hätte die Sidebar keine entsprechende Hintergrundfarbe mehr). Dies entspricht in der Regel der Darstellung auf einem Tablet.

```
1 @media screen and (max-width: 768px) {  
2   nav {  
3     float: none;  
4     width: 100%;  
5     text-align: center;  
6   }  
7   nav ul {  
8     padding: 0;  
9   }  
10  nav ul li {  
11    list-style-type: none;  
12    display: inline;  
13  }  
14  nav ul li:not(:last-child) {  
15    margin-right: 0.5em;  
16  }  
17  main {  
18    width: 100%;  
19  }  
20 }
```

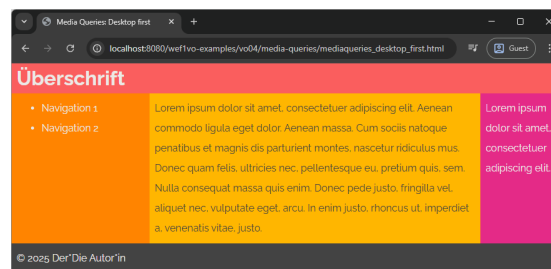
Schließlich wird noch eine Darstellung für Smartphones angestrebt. Da hier weniger horizontaler Platz ist, werden die beiden Inhaltsbereiche und der Hauptbereich nicht mehr gefloatet und auf volle Breite skaliert. Alles ist nun untereinander angeordnet.

```
1 @media screen and (max-width: 480px) {  
2   section#content, aside, main {  
3     float: none;  
4     width: 100%;  
5   }  
6 }
```

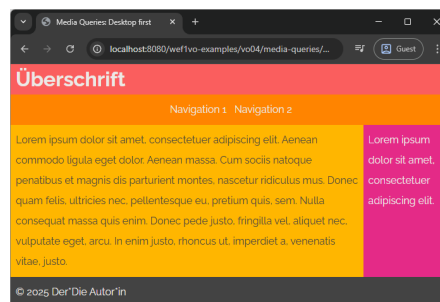
Abbildung 4.20 zeigt alle vier Zustände der Seite.



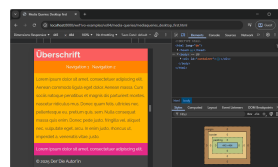
(a)



(b)



(c)



(d)

Abbildung 4.20: Das Layout nach den jeweiligen Breakpoints: bei 1156px Breite (a), bei 810px Breite (b), bei 630px Breite (c) und bei 456px Breite (d). Bei der letzten Einstellung muss der Developer Mode der Chrome DevTools verwendet werden, da aktuelle Chrome-Versionen keine Fenstergrößen unter 500 Pixel zulassen.

Mobile first

Der Mobile first Ansatz funktioniert umgekehrt. Zunächst wird die Seite für eine mobile Auflösung, also für geringe Breite, mit kleineren Grafiken etc. konzipiert. Dann wird in Richtung einer Desktop-Auflösung gearbeitet und es werden überall dort Breakpoints gesetzt, wo das bessere Ausnützen des zusätzlichen horizontalen Platzes, Sinn macht.

Beim Mobile first Ansatz werden daher Medieneigenschaften wie `min-width` verwen-

det, um alles *über* einer bestimmten Auflösung abzuändern.

Das obige Beispiel wird Mobile first wie folgt umgesetzt. Zunächst werden Formatierungen wie Hintergrundfarben und das horizontale Ausrichten der Navigation gesetzt, da alles untereinander angeordnet wird und dies keine besondere Formatierung benötigt:

```

1 div#container {
2   background-color: rgb(255 132 0);
3 }
4 nav {
5   text-align: center;
6 }
7 nav ul {
8   padding: 0;
9 }
10 nav ul li {
11   list-style-type: none;
12   display: inline;
13 }
14
15 nav ul li:not(:last-child) {
16   margin-right: 0.5em;
17 }
18
19 main {
20   background-color: rgb(228 42 135);
21 }
22
23 section#content {
24   background-color: rgb(255 182 0);
25   color: rgb(67 67 67);
26 }
```

Dann wird ein Breakpoint bei 480 Pixeln definiert. Ab dieser Größe sollen die beiden Spalten für den Inhalt wieder nebeneinander stehen. Dazu werden sie zusammen mit dem `<main>`-Element gefloatet. Ebenso werden die Breitenangaben angepasst.

```

1 @media screen and (min-width: 480px) {
2   main {
3     width: 100%;
4     float: right;
5   }
6   section#content {
7     float: left;
8     width: 80%;
9   }
10  aside {
11    float: left;
12    width: 20%;
13  }
14  footer {
15    clear: both;
16  }
17 }
```

In der nächsthöheren Stufe ab 768 Pixeln soll dann auch die Navigation wieder links neben dem Inhalt erscheinen. Also floatet auch sie und erhält angepasste Breitenangaben. Die Navigation erscheint wieder als Liste, weshalb die entsprechenden Angaben

gesetzt werden.

```
1 @media screen and (min-width: 768px) {  
2   nav {  
3     float: left;  
4     width: 25%;  
5     text-align: left;  
6   }  
7   main {  
8     width: 75%;  
9   }  
10  nav ul {  
11    padding-left: 40px;  
12  }  
13  nav ul li {  
14    list-style-type: disc;  
15    display: list-item;  
16  }  
17  nav ul li:not(:last-child) {  
18    margin-right: 0;  
19  }  
20 }
```

Schließlich wird noch ab 960 Pixeln der Container fest auf diese Breite gesetzt.

```
1 @media screen and (min-width: 960px) {  
2   div#container {  
3     width: 960px;  
4     margin: 0 auto;  
5   }  
6 }
```

Damit wurde ein Layout mit demselben Verhalten wie das Desktop first Beispiel geschaffen, Abbildung 4.20 gilt also auch hier. Der Ansatz ist jedoch genau umgekehrt.

Welches das bessere Rezept ist, hängt sicherlich vom Anwendungsfall ab und kann nicht pauschal beantwortet werden. Im Sinne eines steigenden Marktanteils mobiler Endgeräte ist die Mobile first Lösung vielleicht etwas vorzuziehen, zumal sie Entwickler*innen auch dazu zwingt, von einem reduzierten Design und Umfang auszugehen und diese übersichtlich aufzubauen.

4.5 Weiterführende Literatur

Informationen zum CSS Box-Modell und zur Positionierung von Elementen werden in [2, 4–6] ausführlich erläutert.

Selbstverständlich bietet auch die Dokumentation zur CSS Recommendation [7] eine Übersicht, wenngleich diese mitunter sehr technisch und nicht immer ganz einfach zu lesen ausfällt.

Quellenverzeichnis

- [1] Chris Coyier. *All About Floats*. 25. März 2021. URL: <https://css-tricks.com/all-about-floats/> (besucht am 16. 11. 2025) (siehe S. 4-17).
- [2] Paul Fuchs. *HTML5 und CSS3 für Einsteiger. Der leichte Weg zur eigenen Webseite*. Landshut, Deutschland: BMU Verlag, 21. Juni 2019 (siehe S. 4-31).
- [3] Geoff Graham. *Media Queries for Standard Devices*. 12. Aug. 2024. URL: <https://css-tricks.com/snippets/css/media-queries-for-standard-devices/> (besucht am 16. 11. 2025) (siehe S. 4-26).
- [4] Eric Meyer und Estelle Weyl. *CSS. The Definitive Guide*. 5. Aufl. Sebastopol, USA: O'Reilly, 30. Mai 2023 (siehe S. 4-31).
- [5] Peter Müller. *Einstieg in HTML und CSS. Webseiten programmieren und gestalten*. 3. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Sep. 2024 (siehe S. 4-31).
- [6] Jürgen Wolf. *HTML und CSS. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Aug. 2023 (siehe S. 4-31).
- [7] World Wide Web Consortium. *Cascading Style Sheets Level 2 Revision 1 (CSS 2.1) Specification*. W3C Recommendation. Edited in place 12 April 2016 to point to new work, edited in place 07 December 2021 to correct markup. 7. Juni 2011. URL: <https://www.w3.org/TR/CSS2/> (siehe S. 4-1, 4-31).
- [8] World Wide Web Consortium. *CSS Box Model Module Level 3*. W3C Recommendation. 11. Apr. 2024. URL: <https://www.w3.org/TR/css-box-3/> (siehe S. 4-1).
- [9] World Wide Web Consortium. *CSS Box Sizing Module Level 3*. W3C Working Draft. 17. Dez. 2021. URL: <https://www.w3.org/TR/css-sizing-3/> (siehe S. 4-11).
- [10] World Wide Web Consortium. *CSS Positioned Layout Module Level 3*. W3C Working Draft. 7. Okt. 2025. URL: <https://www.w3.org/TR/css-position-3/> (siehe S. 4-12).
- [11] World Wide Web Consortium. *Media Queries Level 3*. W3C Recommendation. 21. Mai 2024. URL: <https://www.w3.org/TR/mediaqueries-3/> (siehe S. 4-22, 4-23, 4-25).
- [12] World Wide Web Consortium. *Media Queries Level 4*. W3C Candidate Recommendation Draft. 25. Dez. 2021. URL: <https://www.w3.org/TR/mediaqueries-4/> (siehe S. 4-22, 4-23, 4-25).

- [13] World Wide Web Consortium. *Media Queries Level 5*. W3C Working Draft. 18. Dez. 2021. URL: <https://www.w3.org/TR/mediaqueries-5/> (siehe S. 4-22, 4-23, 4-24, 4-25).